The Journal of Open Source Software

srafq: Fast, robust fetching of SRA/ENA reads with Aspera-first fallback

Kentaro Miyamoto [1]

1 Research Department, R&D Division, Miyarisan Pharmaceutical Co., Ltd., Saitama, Japan

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 08 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

srafq is a tiny, Linux-only Bash helper that retrieves sequencing reads from SRA/ENA using **Aspera (ascp) when available**, and cleanly falls back to `fasterq-dump` otherwise. It resolves run metadata via ENA over HTTP (metadata only), creates per-accession output directories, and records failures/retries for robust batch processing.

Statement of need

The use of deposited datasets from public archives has grown substantially, while per-run and per-project data volumes have increased dramatically. Consequently, **faster and more reliable download pipelines** are required to avoid bottlenecks in the early stages of analysis. Fetching public read archives reproducibly at scale is often brittle: corporate firewalls may block Aspera (`ascp`), runs mix single/paired layouts, and interrupted transfers leave no clean way to resume. **srafq** addresses this acquisition stage specifically with an Aspera-first strategy, clean fallback to `fasterq-dump`, per-accession output directories, and explicit failure/retry ledgers.

Functionality

- Aspera-first transfers with automatic fallback to `fasterq-dump`.
- Resolves run metadata via the ENA HTTP API (metadata only) (*ENA Portal/Browser API*, n.d.).
- Writes outputs under `OUTDIR/<accession>/` with `_1/_2` naming for paired-end runs.
- Records failures in `data/srafq.failed.tsv` and a retry list at `data/srafq.retry.txt`.
- Minimal dependencies; Linux/Bash 4 only; optional `ascp` (*IBM Aspera FASP*, n.d.), and `sra-tools` (*SRA Toolkit*, n.d.).

Implementation and design

srafq is a single Bash script that prioritizes IBM Aspera `ascp` transfers and falls back to `fasterq-dump` in a failure-tolerant manner. The script (i) resolves run metadata via ENA over HTTP (metadata only), (ii) infers single/paired layout and creates per-accession directories, and (iii) maintains explicit failure (`srafq.failed.tsv`) and retry (`srafq.retry.txt`) ledgers to enable deterministic resumption.

Transfer logic. When `ascp` and its private key are available, *srafq* uses `ascp -QT -k1 -P 33001` with an optional rate cap (`ASCP_LIMIT_M`) and key (`ASCP_KEY`). On `ascp` error or in constrained networks, the script falls back to `fasterq-dump` with user-configurable threads and optional `--skip-technical`. This design targets high-latency environments where Aspera improves throughput but may be blocked or intermittent.

Configuration. Behavior is controlled by environment variables (e.g., `THREADS`, `ASCP_BIN`, `ASCP_KEY`, `ASCP_LIMIT_M`, `RESUME_MODE`, `LAYOUT_MODE`, `SKIP_TECHNICAL`). Defaults are conservative, and the tool is Linux-only by design to keep dependencies minimal.

Failure handling. Each attempt is recorded; failures append one tab-separated line to `srafq.failed.tsv`, and accessions are added to `srafq.retry.txt` for reproducible reruns. A finalize step removes the ledger if no failures remain.

Quality control

We provide continuous integration that runs ShellCheck over the script and a smoke job ensuring `--help` and `--version` execute on each push/PR. Additionally, a tiny **Bats** test verifies these entry points without requiring network access.

Performance evaluation (minimal)

We evaluated end-to-end wall-clock time for representative runs on a typical institutional network in two conditions: (A) Aspera enabled and (B) Aspera disabled (`ASCP_BIN=none`, fallback to `fasterq-dump`). **Measurements reflect a single representative run with `THREADS=2`;** Aspera was rate-limited to `ASCP_LIMIT_M=150m`.

Accession	Aspera (A)	Fallback (B)	Speed-up (A/B)
SRR10479824	00:28:02	03:22:07	×7.21
SRR10479825	00:29:59	03:54:29	×7.82

Reproduction. Example commands:

```
# A) Aspera enabled
export ASCP_KEY="$CONDA_PREFIX/etc/asperaweb_id_dsa.openssh"
ASCP_LIMIT_M=150m THREADS=2 /usr/bin/time -f '%E' \
  ./srafq -i SRR_List.txt -o data_ascp

# B) Fallback (Aspera disabled)
ASCP_BIN=none THREADS=2 /usr/bin/time -f '%E' \
  ./srafq -i SRR_List.txt -o data_fqdump
```

Comparison to related tools

Existing tooling either focuses on downstream processing (e.g., pipelines and QC/QA) or provides single-mode downloaders. *srafq* addresses the acquisition stage specifically with (i) an Aspera-first, fallback-safe strategy, (ii) layout-aware output structuring, and (iii) explicit resume ledgers in a minimal Bash-only footprint. This complements, rather than replaces, downstream analysis pipelines.

Availability

- **Source code**: https://github.com/KentaroMiya/srafq
- **Archive/DOI**: 10.5281/zenodo.17067028 (versioned release archived on Zenodo; corresponding to v0.0.1)
- **License**: MIT

DRAFT

Acknowledgements

We thank the SRA/ENA teams and the maintainers of sra-tools. We also acknowledge IBM Aspera for providing the ascp client implementing the FASP high-speed transfer protocol used by *srafq*. "Aspera" and "FASP" are trademarks of IBM. *srafq* is an independent open-source project and is not affiliated with or endorsed by IBM. Users should obtain and use ascp under its respective license.

References

ENA portal/browser API. (n.d.). https://www.ebi.ac.uk/ena/browser/api/

IBM aspera FASP. (n.d.). https://www.ibm.com/products/aspera

SRA toolkit. (n.d.). https://github.com/ncbi/sra-tools

DRAFT